

INDUSTRIAL PROBLEM SOLVING

1. Telecom Data Plan Recommendation using KNN

```
# =====
# INDUSTRIAL PROBLEM 1
# Telecom Data Plan Recommendation using KNN
# =====

import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# -----
# 1. Create sample customer dataset
# -----

data = {
    "Data_Usage_GB": [2, 3, 4, 5, 6, 7, 8, 9,
                     15, 17, 18, 20, 22, 24, 25, 27,
                     35, 38, 40, 42, 45, 48, 50, 55],

    "Calls_Minutes": [80, 100, 120, 150, 160, 180, 200, 220,
                     150, 170, 190, 210, 220, 230, 250, 260,
                     200, 220, 240, 260, 280, 300, 320, 350],

    "Streaming_Hours": [2, 3, 4, 5, 6, 7, 8, 10,
                       15, 18, 20, 22, 25, 27, 30, 32,
                       40, 42, 45, 48, 50, 55, 60, 65],

    "Connected_Devices": [1, 1, 1, 2, 2, 2, 2, 2,
                          2, 2, 3, 3, 3, 3, 3, 3,
                          4, 4, 4, 4, 5, 5, 5, 5],

    "Plan": [
        "Basic", "Basic", "Basic", "Basic",
        "Basic", "Basic", "Basic", "Basic",

        "Standard", "Standard", "Standard", "Standard",
        "Standard", "Standard", "Standard", "Standard",

        "Premium", "Premium", "Premium", "Premium",
        "Premium", "Premium", "Premium", "Premium"
    ]
}

df = pd.DataFrame(data)

print("Customer Dataset")
print("-----")
print(df)

# -----
# 2. Select features and target
# -----

features = [
    "Data_Usage_GB",
    "Calls_Minutes",
    "Streaming_Hours",
    "Connected_Devices"
]

X = df[features]
y = df["Plan"]

# -----
# 3. Split data into training and testing
# -----
```

```

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y
)

# -----
# 4. Normalize the features
# -----

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# -----
# 5. Ask user for K
# -----

k = int(input("Enter value of K (example: 3): "))

# -----
# 6. Train KNN model
# -----

model = KNeighborsClassifier(n_neighbors=k)

model.fit(X_train_scaled, y_train)

# -----
# 7. Test the model
# -----

y_pred = model.predict(X_test_scaled)

accuracy = accuracy_score(y_test, y_pred)

print("\nKNN Classification Results")
print("-----")

print("Actual Classes      :", list(y_test))
print("Predicted Classes :", list(y_pred))

print("\nAccuracy:",
      round(accuracy * 100, 2), "%")

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# -----
# 8. New customer recommendation
# -----

print("\nEnter New Customer Details")
print("-----")

data_usage = float(input("Monthly Data Usage (GB): "))
calls = float(input("Monthly Call Minutes: "))
streaming = float(input("Streaming Hours: "))
devices = int(input("Number of Connected Devices: "))

new_customer = pd.DataFrame([[
    data_usage,
    calls,
    streaming,
    devices
]], columns=features)

# Scale new customer
new_customer_scaled = scaler.transform(new_customer)

# Predict plan

```

```

recommended_plan = model.predict(new_customer_scaled)[0]

print("\nRecommended Data Plan:", recommended_plan)

# -----
# 9. Display nearest customers
# -----

distances, indices = model.kneighbors(
    new_customer_scaled
)

print("\nNearest Customers")
print("-----")

for i in range(k):

    index = indices[0][i]

    print(
        "Customer:", df.iloc[index]["Plan"],
        "| Distance:",
        round(distances[0][i], 3)
    )

```

Customer Dataset

```

-----
      Data_Usage_GB  Calls_Minutes  Streaming_Hours  Connected_Devices  Plan
0                2             80              2              1  Basic
1                3            100              3              1  Basic
2                4            120              4              1  Basic
3                5            150              5              2  Basic
4                6            160              6              2  Basic
5                7            180              7              2  Basic
6                8            200              8              2  Basic
7                9            220             10              2  Basic
8               15            150             15              2  Standard
9               17            170             18              2  Standard
10              18            190             20              3  Standard
11              20            210             22              3  Standard
12              22            220             25              3  Standard
13              24            230             27              3  Standard
14              25            250             30              3  Standard
15              27            260             32              3  Standard
16              35            200             40              4  Premium
17              38            220             42              4  Premium
18              40            240             45              4  Premium
19              42            260             48              4  Premium
20              45            280             50              5  Premium
21              48            300             55              5  Premium
22              50            320             60              5  Premium
23              55            350             65              5  Premium

```

Enter value of K (example: 3): 3

KNN Classification Results

```

-----
Actual Classes   : ['Standard', 'Premium', 'Basic', 'Premium', 'Basic', 'Standard']
Predicted Classes : ['Standard', 'Premium', 'Basic', 'Premium', 'Basic', 'Standard']

```

Accuracy: 100.0 %

Confusion Matrix:

```

[[2 0 0]
 [0 2 0]
 [0 0 2]]

```

Classification Report:

	precision	recall	f1-score	support
Basic	1.00	1.00	1.00	2
Premium	1.00	1.00	1.00	2
Standard	1.00	1.00	1.00	2
accuracy			1.00	6
macro avg	1.00	1.00	1.00	6
weighted avg	1.00	1.00	1.00	6

Enter New Customer Details

```

-----
Monthly Data Usage (GB): 2
Monthly Call Minutes: 120

```

2. Electricity Consumption Prediction using Locally Weighted Regression

```

# =====
# INDUSTRIAL PROBLEM 2
# Electricity Consumption Prediction
# using Locally Weighted Regression
# =====

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score

# -----
# 1. Generate sample electricity dataset
# -----

np.random.seed(42)

n = 500

# Time of day: 0 to 23 hours
hour = np.random.randint(0, 24, n)

# Temperature varies throughout the day
temperature = (
    25
    + 7 * np.sin((hour - 6) * np.pi / 12)
    + np.random.normal(0, 1.5, n)
)

# Humidity
humidity = np.random.uniform(40, 85, n)

# Number of active appliances
appliances = np.random.randint(1, 8, n)

# -----
# 2. Generate realistic electricity consumption
# -----

# Base consumption
consumption = (
    1.5
    + 0.08 * temperature
    + 0.04 * humidity
    + 0.6 * appliances
)

# Morning peak
consumption += np.where(
    (hour >= 6) & (hour <= 9),
    2.0,
    0
)

# Evening peak
consumption += np.where(
    (hour >= 18) & (hour <= 22),
    3.5,
    0
)

# Night reduction
consumption -= np.where(
    (hour >= 0) & (hour <= 5),
    1.0,
    0
)

# Add random noise
noise = np.random.normal(0, 1.0, n)

```

```

consumption += noise

# Ensure positive consumption
consumption = np.maximum(consumption, 0.5)

# -----
# 3. Create DataFrame
# -----

df = pd.DataFrame({
    "Hour": hour,
    "Temperature": temperature,
    "Humidity": humidity,
    "Appliances": appliances,
    "Consumption_kWh": consumption
})

# Sort according to hour
df = df.sort_values("Hour").reset_index(drop=True)

print("Electricity Consumption Dataset")
print("-----")
print(df.head(10))

# -----
# 4. Feature matrix
# -----

features = [
    "Hour",
    "Temperature",
    "Humidity",
    "Appliances"
]

X = df[features].values
y = df["Consumption_kWh"].values

# -----
# 5. Normalize features
# -----

X_mean = X.mean(axis=0)
X_std = X.std(axis=0)

X_scaled = (X - X_mean) / X_std

# -----
# 6. Locally Weighted Regression Function
# -----

def locally_weighted_regression(
    X_train,
    y_train,
    X_query,
    tau=0.8
):
    predictions = []

    # Add bias column
    X_train_bias = np.c_[
        np.ones(len(X_train)),
        X_train
    ]

    for x in X_query:
        # Add bias
        x_bias = np.r_[1, x]

        # Calculate squared distances
        distances = np.sum(
            (X_train - x) ** 2,
            axis=1
        )

```

```

        # Gaussian kernel weights
        weights = np.exp(
            -distances / (2 * tau ** 2)
        )

        # Weight matrix
        W = np.diag(weights)

        # Calculate theta
        try:
            theta = np.linalg.pinv(
                X_train_bias.T @ W @ X_train_bias
            ) @ (
                X_train_bias.T @ W @ y_train
            )

        except np.linalg.LinAlgError:
            theta = np.linalg.pinv(
                X_train_bias.T @ X_train_bias
            ) @ (
                X_train_bias.T @ y_train
            )

        # Prediction
        prediction = x_bias @ theta

        predictions.append(prediction)

    return np.array(predictions)

# -----
# 7. Split data
# -----

split = int(0.8 * len(X_scaled))

X_train = X_scaled[:split]
X_test = X_scaled[split:]

y_train = y[:split]
y_test = y[split:]

# -----
# 8. Train / Predict using local regression
# -----

tau = 0.8

y_pred = locally_weighted_regression(
    X_train,
    y_train,
    X_test,
    tau
)

# -----
# 9. Evaluate model
# -----

mae = mean_absolute_error(
    y_test,
    y_pred
)

rmse = np.sqrt(
    mean_squared_error(
        y_test,
        y_pred
    )
)

r2 = r2_score(
    y_test,

```

```

        y_pred
    )

print("\nLocally Weighted Regression Results")
print("-----")

print("Bandwidth (tau):", tau)

print("MAE :", round(mae, 3), "kWh")
print("RMSE:", round(rmse, 3), "kWh")
print("R²  :", round(r2, 3))

# -----
# 10. Display predictions
# -----

results = pd.DataFrame({
    "Actual Consumption": y_test,
    "Predicted Consumption": y_pred
})

print("\nSample Predictions")
print("-----")
print(results.head(15))

# -----
# 11. Plot actual vs predicted
# -----

plt.figure(figsize=(10, 5))

plt.plot(
    y_test,
    label="Actual Consumption"
)

plt.plot(
    y_pred,
    label="Predicted Consumption"
)

plt.xlabel("Test Sample")
plt.ylabel("Electricity Consumption (kWh)")
plt.title(
    "Actual vs Predicted Electricity Consumption"
)

plt.legend()
plt.grid(True)

plt.show()

# -----
# 12. Plot consumption according to time
# -----

plt.figure(figsize=(10, 5))

plt.scatter(
    df["Hour"],
    df["Consumption_kWh"],
    alpha=0.5
)

plt.xlabel("Hour of Day")
plt.ylabel("Electricity Consumption (kWh)")

plt.title(
    "Electricity Consumption vs Time"
)

plt.grid(True)

plt.show()
# =====
# Effect of Bandwidth (Tau)
# =====

```

```
tau_values = [0.1, 0.3, 0.5, 0.8, 1.0, 2.0]

print("Bandwidth Comparison")
print("-----")

for tau in tau_values:

    predictions = locally_weighted_regression(
        X_train,
        y_train,
        X_test,
        tau
    )

    mae = mean_absolute_error(
        y_test,
        predictions
    )

    rmse = np.sqrt(
        mean_squared_error(
            y_test,
            predictions
        )
    )

    r2 = r2_score(
        y_test,
        predictions
    )

    print(
        "Tau =", tau,
        "| MAE =", round(mae, 3),
        "| RMSE =", round(rmse, 3),
        "| R2 =", round(r2, 3)
    )
```


Electricity Consumption Dataset

	Hour	Temperature	Humidity	Appliances	Consumption_kWh
0	0	16.934808	55.294373	5	7.335152
1	0	16.494363	69.649846	7	8.677005
2	0	15.848453	58.310858	2	6.269768
3	0	20.355742	49.705382	3	5.708558
4	0	17.014150	82.831530	6	7.193203
5	0	17.775183	73.138091	5	7.929978
6	0	19.278370	45.920546	7	9.054289
7	0	16.983651	45.889315	1	4.417469
8	0	18.270796	72.722608	4	6.335129
9	0	19.448883	72.921528	2	6.686680

Locally Weighted Regression Results

Bandwidth (tau): 0.8
MAE : 2.211 kWh
RMSE: 3.056 kWh
R² : -1.037

Sample Predictions

	Actual Consumption	Predicted Consumption
0	12.698211	12.435592
1	9.612028	10.694725
2	8.095279	9.098739
3	12.777842	11.021709
4	11.650547	10.876066
5	13.717636	12.580114
6	11.475301	12.040730
7	13.458323	12.501622
8	10.304208	9.894037
9	9.920776	10.008228
10	13.654483	12.474852
11	10.416656	9.780384
12	11.375329	11.522852
13	11.625566	12.005515